Supermarket Database Management System

Duygu Akyıldız
Student ID: 220303032

January 2026

Abstract

This report presents the completed implementation of a **Supermarket Database Management System** designed to model real-world retail operations in a multi-branch environment. The system manages branches, products, categories, suppliers, customers, sales transactions, and inventory movements within a fully normalized relational structure.

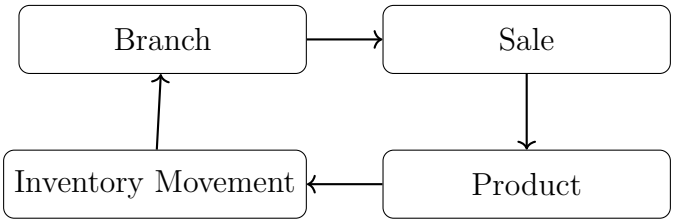
The database is implemented using **MySQL** and enforces strong data integrity through primary and foreign key constraints. Inventory consistency is preserved using transaction-based movement records, ensuring accurate stock tracking across all branches. Sample data and test queries confirm reliable traceability from supplier delivery to customer sales.

1 Introduction

Modern supermarket systems must handle high transaction volumes while maintaining accurate inventory and sales records across multiple branches. Inconsistent data structures may lead to stock mismatches, reporting errors, and operational inefficiencies.

The objective of this project is to design and implement a robust relational database system that reflects real-world supermarket workflows. The proposed design applies normalization principles, Enhanced Entity–Relationship (EER) modeling, and transactional integrity rules to support reliable and scalable retail operations.

Figure 1: Conceptual Overview of the Supermarket Database System



2 Procedure

The development of the Supermarket Database Management System followed a structured workflow:

1. Analysis of supermarket business processes such as sales, stock replenishment, and customer transactions.
2. Identification of core entities including Branch, Product, Customer, Supplier, Sale, and Inventory Movement.
3. Construction of an Enhanced Entity–Relationship (EER) diagram.
4. Transformation of the EER model into a normalized relational schema (Third Normal Form).
5. Implementation of tables, constraints, and sample data in MySQL.
6. Validation of the database using test queries and integrity checks.

3 Theory

The database is designed according to relational database principles and normalized to **Third Normal Form (3NF)**. The system follows an event-based modeling approach to ensure complete traceability of inventory changes.

Master entities represent relatively stable data such as BRANCH, PRODUCT, CATEGORY, SUPPLIER, and CUSTOMER. Operational entities such as SALE, SALE_ITEM, and INVENTORY_MOVEMENT capture all time-dependent business activities.

3.1 Inventory Management Design

Instead of updating stock quantities directly within product records, the system uses a branch-level stock table combined with an inventory movement log. All stock changes (inbound from suppliers or outbound via sales) are recorded as movement events. This design guarantees data consistency and supports historical auditability.

4 Results

The database schema was successfully implemented in MySQL with all integrity constraints enforced. Sample data were inserted and tested to validate entity relationships and transaction flows.

Entity Overview

Entity	Purpose	Relationship Type
Branch	Store locations	1–N Sales
Product	Sale items	N–N via Sale_Item
Customer	Buyers	Optional participation
Supplier	Product providers	1–N Supplies
Sale	Sales transactions	Header entity
Inventory_Movement	Stock history	Event entity

5 Remarks and Recommendations

The implemented Supermarket Database Management System provides a consistent and scalable foundation for multi-branch retail operations. The system already supports sales processing, inventory tracking, supplier management, and discount mechanisms using a normalized relational structure.

Possible future enhancements that may further strengthen the system include:

- User and authorization management
- Performance optimization
- Advanced reporting
- Real-time POS integration

References

1. Elmasri, R., Navathe, S. B. *Fundamentals of Database Systems*.
2. Silberschatz, A., Korth, H. F., Sudarshan, S. *Database System Concepts*.
3. MySQL Documentation.
4. dbdiagram.io.